Table de hachage en adressage ouvert

Le but de ce TP est de construire une implémentation efficace de la structure de donnée impérative « ensemble » (set en anglais). Elle permet de manipuler des ensembles de valeurs de type `T`. La spécification d'une telle structure est donnée ci-dessous.

```
set *set_new(void);
bool set_is_member(set *s, T x);
void set_add(set *s, T x);
void set_remove(set *s, T x);
void set_delete(set *s);
```

Ces fonctions nous permettent de créer un ensemble vide, de savoir si un élément x fait partie de l'ensemble s, d'ajouter ou d'enlever un élément à notre ensemble et enfin de libérer la mémoire utilisée par s. On peut imaginer l'utilisation d'une telle structure pour gérer l'ensemble des adresses IP bannies d'un réseau pour des raisons de sécurité. Les adresses IP étant codées sur 32 bits, dans la suite de ce TP, nous utiliserons `T = uint32_t`, le symbole `T` étant simplement utilisé dans l'énoncé comme un raccourci.

Nous allons utiliser une implémentation utilisant une *table de hachage en adressage ouvert*. Cette structure fonctionne à l'aide d'un tableau de taille 2^p (où $p \in [\![1, 63]\!]$ est amené à évoluer au cours de la durée de vie de la structure) ainsi qu'une fonction de signature

```
uint64_t hash(T x, int p);
```

appelée *fonction de hachage*. À tout élément x de type `T` et tout entier $p \in \mathbb{N}$, elle associe un indice de tableau $i \in [\![0, 2^p[\![$ dans lequel nous souhaitons placer l'élément x. La fonction de hachage le plus simple à notre disposition est définie par

$$\forall x \in \mathbb{Z}, \quad \mathrm{hash}_p(x) := x \bmod 2^p.$$

C'est celle que nous utiliserons dans la première partie de ce TP. Si $p = 2$, en partant d'une table de hachage vide, l'ajout des valeurs $x = 1492$ et $x = 1515$ dont les hachages respectifs sont $\mathrm{hash}_2(1492) = 0$ et $\mathrm{hash}_2(1515) = 3$, aboutira au tableau suivant :

1492			1515
0	1	2	3

En suivant, cette stratégie, il est alors facile de voir que 1515 est présent dans notre tableau : il suffit de calculer son hachage $\mathrm{hash}_2(1515) = 3$ et de constater que l'élément 1515 est bien dans la case d'indice 3.

Le rôle d'une bonne fonction de hachage est de répartir le plus uniformément possible les éléments de type `T` dans les différentes cases du tableau. Malheureusement, il est possible que des valeurs x et y soient différentes tout en ayant $\mathrm{hash}_p(x) = \mathrm{hash}_p(y)$; on parle alors de *collision*. Imaginons un instant que l'élément x ait déjà été placé dans le tableau à la case $\mathrm{hash}_p(x)$. Il nous est alors impossible de placer y dans la même case. La stratégie d'une table de hachage en « adressage ouvert » consiste à le placer dans une case adjacente en suivant la stratégie décrite dans le paragraphe suivant.

Tout d'abord, afin de savoir si une case du tableau est vide ou occupée, nous allons les marquer d'une couleur. Sur nos schémas, les cases seront par défaut de couleur jaune pour signifier qu'elles sont « libres ». L'ajout et la recherche d'un élément se passe alors de la manière suivante.

- *Ajout d'un élément* : Lorsqu'on souhaite ajouter l'élément x dans notre tableau, on commence par calculer son hachage $i := \mathrm{hash}_p(x)$.
  - Si la case d'indice i est libre, on y stocke l'élément x et on la colorie en bleu pour signifier qu'elle est « occupée ».
  - Si cette case n'est pas libre, nous allons tester successivement les cases d'indices $i + 1 \bmod 2^p$, $i + 2 \bmod 2^p$, $i + 3 \bmod 2^p, \ldots$ jusqu'à trouver une case qui est libre ; on parle de *sondage linéaire*. Dès qu'une telle case est trouvée, on y place l'élément x et on la colorie en bleu pour signifier qu'elle est « occupée ».

Bien entendu, un adressage ouvert suppose que le nombre d'éléments présents dans le tableau est toujours inférieur ou égal à 2^p. Pour simplifier la recherche, on imposera de plus que le tableau contienne toujours au moins une case « libre ». Lorsque l'occupation du tableau deviendra trop grande, il sera nécessaire de le redimensionner ; sa taille sera alors doublée.

— *Recherche d'un élément* : Lorsqu'on cherche la présence d'un élément x, il suffit de calculer son hachage $i := \mathrm{hash}_p(x)$ et de chercher, par sondage linéaire, la présence de x à partir de l'indice i.
  — Si au cours de la recherche, on trouve une case « libre », c'est que l'élément n'est pas présent.
  — Si la recherche passe par une case « occupée » contenant l'élément x, c'est qu'il est présent dans la table.

Afin de mieux comprendre notre stratégie, en partant d'un tableau de taille 4 initialement vide, après les opérations :
— ajout de l'élément 1492 dont le hachage est $\mathrm{hash}_2(1492) = 0$,
— ajout de l'élément 1515 dont le hachage est $\mathrm{hash}_2(1515) = 3$,
— ajout de l'élément 1939 dont le hachage est $\mathrm{hash}_2(1939) = 3$,
voici l'état de notre table de hachage.

1492	1939		1515
0	1	2	3

Les éléments 1492, 1515 sont tout d'abord placés dans les cases vides données par leur hachage. L'élément 1939 a pour hachage 3. Puisque cette case est déjà « occupée », on va sonder les cases suivantes de manière circulaire jusqu'à en trouver une de « libre ». C'est la case d'indice $j := 1$ qui est trouvée.

Pour prendre en compte la couleur de chacune de nos cases, nous allons utiliser un « statut » qui peut prendre deux valeurs :
— `empty = 0`, pour signifier que la case est « libre ».
— `occupied = 1`, pour signifier que la case est « occupée ».
Notre table sera donc formée d'un tableau de « sceaux » (bucket en anglais), chacun composé d'un statut et d'un élément. Nous utiliserons donc les structures suivantes

```
const uint8_t empty = 0;
const uint8_t occupied = 1;

struct bucket {
    uint8_t status;
    T element;
};
typedef struct bucket bucket;

struct set {
    int p;
    bucket *a;
    uint64_t nb_empty;
};
typedef struct set set;
```

où $p \in [\![1, 63]\!]$ et le tableau `a` est de taille 2^p. L'entier `nb_empty` contiendra le nombre de cases « libres » du tableau.

1 Constructeur, destructeur et recherche d'éléments

1. Écrire la fonction `hash` implémentant le hachage $\mathrm{hash}_p(x) := x \bmod 2^p$.

   ```
   uint64_t hash(T x, int p);
   ```

   Afin de proposer une implémentation efficace, on utilisera les opérations bit à bit disponibles en C.

2. Écrire la fonction `set *set_new(void)` créant une table de hachage pour laquelle $p = 1$, dont toutes les cases sont « libres ». Afin de pouvoir tester plus facilement nos prochaines fonctions, on écrira aussi une fonction `set *set_example(void)` générant artificiellement une table de hachage pour laquelle $p = 2$ et contenant les dates 1492, 1515 et 1939 comme décrit dans l'exemple plus haut.

3. Écrire la fonction `void set_delete(set *s)` permettant de libérer la mémoire utilisée par `s`.

4. Écrire la fonction `bool set_is_member(set *s, T x)` permettant de déterminer si x est un élément de s. Afin de proposer une implémentation efficace, on utilisera les opérations bit à bit pour les calculs modulo 2^p. On donnera de plus un argument justifiant la terminaison de cette fonction.

2 Parcourt de la table

Afin de parcourir la table de manière efficace, nous allons utiliser un itérateur qui va prendre successivement les indices des cases « occupées » de notre tableau. Il commencera à l'index i_{begin} qui est égal, soit au plus petit index i tel que `a[i]` est occupé (si une telle valeur existe), soit à 2^p. Il terminera par la valeur $i_{\text{end}} := 2^p$. Si i est l'indice d'une case occupée, la fonction `set_next(s, i)` renverra, soit l'indice de la prochaine case occupée, si une telle case existe, soit 2^p.

```
uint64_t set_begin(set *s);
uint64_t set_end(set *s);
uint64_t set_next(set *s, uint64_t i);
```

Nous utiliserons aussi la fonction `set_get(s, i)` qui renvoie l'élément contenu dans la case d'index i (i désignant bien évidemment un index de case occupée).

```
T set_get(set *s, uint64_t i);
```

Ces fonctions permettent de parcourir efficacement l'ensemble des valeurs de notre table de hachage. Par exemple, la fonction suivante permet de savoir si tous les éléments de la table sont pairs.

```
bool all_even(set *s) {
    for (uint64_t i = set_begin(s); i != set_end(s); i = set_next(s, i)) {
        if (set_get(s, i) % 2 == 1) {
            return false;
        }
    }
    return true;
}
```

1. Écrire la fonction `set_get`.
2. Écrire les fonctions `set_begin`, `set_end` et `set_next`.

3 Ajout d'éléments

Afin de préparer la possibilité d'ajouter des éléments à notre table, nous allons factoriser notre code et écrire une fonction

```
uint64_t set_search(set *s, T x, bool *found);
```

qui renvoie un entier i et qui écrit un booléen dans `*found`. Ces valeurs doivent posséder les caractéristiques suivantes :

— Si x est un élément de s, l'entier i renvoyé est l'index de la case contenant x et `*found` est égal à `true`.
— Sinon, on calcule l'index i de la case dans laquelle on placerait x si on avait à l'ajouter à s. On renvoie alors i et `*found` est égal à `false`.

1. Écrire `set_search` et réimplémenter `set_is_member` à l'aide de cette nouvelle fonction.
2. Écrire la fonction `void set_resize(set *s, int p)` prenant en entrée une table de hachage possédant n éléments et « redimensionnant » son tableau en un tableau de taille 2^p. On supposera que $n < 2^p$ et on placera tous les éléments dans ce nouveau tableau en utilisant la fonction de hachage hash_p associée à cette nouvelle taille de tableau. On pourra commencer par créer une nouvelle table de hachage, avant de modifier s.
3. Écrire la fonction `void set_add(set *s, T x)` ajoutant l'élément x à la table s. Afin de toujours conserver une case « libre » dans notre tableau et de ne pas trop le charger, on décidera de doubler la taille du tableau dès que le nombre de cases « libres » est inférieur au tiers de la taille du tableau.

4 Suppression d'éléments

On souhaite désormais pouvoir supprimer des éléments de notre table de hachage.

1. Expliquer pourquoi il n'est pas possible de supprimer un élément de la table en changeant simplement le statut de sa case en case « libre ».

Afin de remédier à ce problème, nous allons créer un nouveau statut appelé « pierre tombale » (tombstone en anglais). Lorsqu'un élément de la table sera supprimé, le statut de sa case passera d' « occupé » à celui de « pierre tombale ». Lors du sondage intervenant dans la recherche d'un élément, il faudra considérer les pierres tombales comme des cases ne contenant aucun élément mais signalant que la recherche doit continuer. Si nous cherchons une case pour y placer un nouvel élément, il faudra déterminer la première case qui est soit libre, soit une pierre tombale. On ajoute donc le statut

```
const uint8_t tombstone = 2;
```

que nous représenterons sur nos schémas par le rose. Par exemple, en partant d'un tableau de taille 4 ayant toutes ses cases « libres », après les opérations suivantes

— ajout de l'élément 1492 dont le hachage est $\mathrm{hash}_2(1492) = 0$,
— ajout de l'élément 1515 dont le hachage est $\mathrm{hash}_2(1515) = 3$,
— ajout de l'élément 1939 dont le hachage est $\mathrm{hash}_2(1939) = 3$,
— suppression de l'élément 1492.

voici l'état de notre table de hachage :

1492	1939		1515
0	1	2	3

2. Proposer des nouvelles implémentations des fonctions `set_search`, `set_begin`, `set_next` et `set_add` fonctionnant avec les pierres tombales.
3. Implémenter la fonction

```
void set_remove(set *s, T x);
```

permettant d'enlever l'élément x de la table s.

5 La liste des adresses IP

Dans un réseau, chaque ordinateur possède une unique adresse nommée *adresse IP*. Le standard IPv4 définit une telle adresse comme un entier non signé 32 bits, généralement représenté sous forme de sa décomposition en base 256 où les « chiffres » sont séparés par des points : $d_3.d_2.d_1.d_0$ où $d_k \in [\![0, 256[\![$. Par exemple, le site web des Lazaristes est hébergé à l'adresse `51.38.180.91`. Le fichier `ip.txt` contient une liste de 172 754 adresses IP que nous souhaitons charger dans notre notre table de hachage.

1. Écrire une fonction

```
T *read_data(char *filename, int *n, int *error);
```

lisant un fichier texte d'adresses IP encodées sous la forme $d_3.d_2.d_1.d_0$. Au retour de la fonction, l'entier pointé par `error` sera nul si le fichier a été trouvé et contiendra la valeur 1 si une erreur s'est produite. L'entier pointé par `n` contiendra la taille du tableau renvoyé.

2. Écrire une fonction

```
void set_skip_stats(set *s, double *average, uint64_t *max);
```

calculant le nombre de moyen et le nombre maximum de sondages nécessaires pour trouver une adresse x appartenant à s. En observant le fichier des adresses IP, expliquer pourquoi ces nombres sont si élevés.

6 Une meilleure fonction de hachage

Afin de résoudre le problème soulevé dans la partie précédente, nous allons implémenter une fonction de hachage plus efficace. Pour cela, on choisit un réel $\varphi \in [0, 1]$ et on définit hash_p par

$$\forall x \in \mathbb{Z}, \quad \mathrm{hash}_p(x) := \lfloor 2^p \{x\varphi\} \rfloor$$

où $\{a\} \coloneqq a - \lfloor a \rfloor$ désigne la partie fractionnaire de $a \in \mathbb{R}$. Même si cette méthode fonctionne quelle que soit la valeur de φ, on peut montrer que lorsque φ est proche de

$$\frac{\sqrt{5}-1}{2} \approx 0.618034$$

la fonction hash_p va favoriser la répartition uniforme des valeurs x dans notre tableau (voir les théorèmes d'ergodicité sur l'équirépartition modulo 1). Nous utiliserons donc une approximation de $(\sqrt{5}-1)/2$ de la forme $\varphi \coloneqq s/2^{64}$ où $s \in \mathbb{N}$.

1. Montrer que la fonction

```
uint64_t f(uint64_t x, uint64_t s) {
    return x * s;
}
```

   calcule l'entier $x_s \in [\![0, 2^{64}[\![$ tel que
$$\{x\varphi\} = \frac{x_s}{2^{64}}.$$

2. Montrer que la décomposition en base 2 de $\mathrm{hash}_p(x)$ est formée des p bits de poids forts de la décomposition en base 2 de x_s.
3. En déduire une implémentation

```
uint64_t hash(uint32_t x, int p)
```

   permettant de calculer efficacement $\mathrm{hash}_p(x)$. On utilisera la valeur
$$s \coloneqq 11\ 400\ 714\ 819\ 323\ 198\ 549$$
   qui a été choisie pour être un nombre premier et pour que $s/2^{64}$ soit une bonne approximation de $(\sqrt{5}-1)/2$.
4. Quel est le nombre moyen et le nombre maximum de sondages nécessaires pour trouver une adresse IP présente dans notre base avec cette nouvelle fonction de hachage ?

7 Sondage quadratique

Afin de faire encore baisser le nombre de sondages nécessaires pour trouver un élément dans notre table, nous allons changer la technique de sondage. Au lieu de sonder les cases d'indices $i+1 \bmod 2^p$, $i+2 \bmod 2^p$, $i+3 \bmod 2^p, \ldots$ nous allons sonder les cases d'indices $i+1 \bmod 2^p$, $i+(1+2) \bmod 2^p$, $i+(1+2+3) \bmod 2^p, \ldots$ afin d'éviter la formation de clusters qui ont tendance à apparaitre avec la technique de sondage linéaire. Cette méthode est appelée méthode de sondage quadratique.

1. Implémenter cette nouvelle méthode et observer son influence sur les nombres de sondage à effectuer pour notre ensemble d'adresses IP.
2. Prouver enfin que cette méthode de sondage est correcte, c'est-à-dire que si il existe une case libre dans notre tableau, le sondage finira par la trouver.